ZimPrep Architecture Audit Report

Date: 2024-12-19
Auditor: Senior Principal Software Architect
System: ZimPrep Production-Grade SaaS Platform
Architecture: Engine-Based with Central Orchestrator

SECTION 1: High-Level Status

Overall Maturity Level: **65%**

Breakdown:

- Core Infrastructure: 85% (Orchestrator, Gateway, Registry)
- Engine Implementation: 70% (Most engines exist, some incomplete)
- Engine Isolation: 90% (Mostly compliant, one violation found)
- Traceability: 75% (Context exists, propagation incomplete)
- RAG Marking Flow: 80% (Implemented but validation incomplete)
- Data Storage: 60% (MongoDB used, SQL separation unclear)
- Async/Background: 70% (Infrastructure exists, integration incomplete)

Can This System Be Safely Extended? **NO**

Critical Blockers:

1. **Pipeline-Registry Mismatch**: Pipelines reference `"identity"` but registry only has `"identity_subscription"`. System will fail on startup for pipelines using `"identity"`.
2. **Engine Isolation Violation**: `practice_assembly` engine has commented-out orchestrator call (lines 113-122), indicating intent to violate isolation.
3. **Missing Engine Registration**: `"identity"` engine referenced in 3 pipelines but not registered.
4. **Incomplete Trace Propagation**: `trace_id` generated in gateway but not consistently propagated to all engines via ExecutionContext.
5. **Validation Veto Implementation**: Validation engine can veto, but orchestrator check may not catch all invalid states.

System is NOT production-ready until these blockers are resolved.

SECTION 2: Implemented Components

2.1 API Gateway Layer

Files: `backend/app/api/gateway.py`, `backend/app/api/dependencies/auth.py`

What Works:

- ☑ JWT authentication via `get_current_user` dependency
- ☑ Pipeline execution endpoint (`/api/v1/pipeline/execute`)

- ☑ RBAC enforcement at gateway level
- ☑ Trace context creation (`ExecutionContext.create()`)
- ☑ Appeal reconstruction endpoint
- ☑ Error handling with proper HTTP status codes

Architecturally Correct:

- ☑ Gateway creates trace_id (not orchestrator)
- ☑ Gateway validates authentication before orchestrator
- ☑ Gateway forwards to orchestrator only (no direct engine calls)
- ☑ Gateway enforces role-based access control

Issues:

- ⚠ Feature flags snapshot not populated from actual service
- ⚠ `request_source` hardcoded to "api" instead of extracted from headers

2.2 Engine Orchestrator

Files: `backend/app/orchestrator/orchestrator.py`, `backend/app/orchestrator/pipelines.py`, `backend/app/orchestrator/engine_registry.py`

What Works:

- ☑ Centralized engine execution via `execute_pipeline()`
- ☑ Pipeline definitions with immutable engine sequences
- ☑ Engine registry with fail-fast validation on startup
- ☑ Appeal integrity enforcement (blocks AI engines during appeals)
- ☑ Reporting integrity enforcement (blocks AI engines during reporting)
- ☑ Validation veto enforcement (aborts pipeline if validation fails)
- ☑ Polymorphic execution (handles both async and sync engines)
- ☑ Fail-fast on engine failures
- ☑ Execution timing and observability

Architecturally Correct:

- ☑ Orchestrator is the ONLY execution authority
- ☑ Engines cannot call each other directly
- ☑ Pipeline order is immutable
- ☑ Engine outputs aggregated immutably
- ☑ Appeal pipelines block AI engines (HARD FAIL)
- ☑ Reporting pipelines block AI engines (HARD FAIL)

Issues:

- ⚠ Deprecated `execute()` method still exists (backward compatibility only)
- ⚠ Identity engine name mismatch (see Section 5)

2.3 Core Engines

2.3.1 Exam Structure Engine

Files: `backend/app/engines/exam_structure/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Fetches exam structure from MongoDB
- ☑ Returns subjects, papers, schedules
- ☑ Proper error handling
- ☑ Returns EngineResponse

2.3.2 Session Timing Engine

Files: `backend/app/engines/session_timing/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Manages exam session timing
- ☑ Validates time limits
- ☑ Returns EngineResponse

2.3.3 Question Delivery Engine

Files: `backend/app/engines/question_delivery/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Delivers questions to students
- ☑ Enforces delivery rules
- ☑ Returns EngineResponse

2.3.4 Submission Engine

Files: `backend/app/engines/submission/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Captures student submissions
- ☑ Validates submission format
- ☑ Returns EngineResponse

2.3.5 Results Engine

Files: `backend/app/engines/results/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Stores final exam results
- ☑ Provides read-only access to results
- ☑ Returns EngineResponse

2.3.6 Identity & Subscription Engine

Files: `backend/app/engines/identity_subscription/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Resolves user identity
- ☑ Checks subscription entitlements
- ☑ Enforces feature flags
- ☑ Returns EngineResponse with entitlement snapshot

Issue:

- ✕ Registered as `"identity_subscription"` but pipelines reference `"identity"`

2.4 AI Engines

2.4.1 Embedding Engine

Files: `backend/app/engines/ai/embedding/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Transforms student answers to vector embeddings
- ☑ Uses OpenAI embeddings API
- ☑ Returns EngineResponse with embedding vector
- ☑ Proper error handling
- ☑ Does NOT call other engines

2.4.2 Retrieval Engine

Files: `backend/app/engines/ai/retrieval/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Retrieves marking evidence from vector store
- ☑ Uses MongoDB Atlas Vector Search
- ☑ Filters by subject, syllabus, paper, question
- ☑ Returns EngineResponse with evidence pack
- ☑ Does NOT call other engines

2.4.3 Reasoning & Marking Engine

Files: `backend/app/engines/ai/reasoning_marking/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Compares student answers to retrieved evidence
- ☑ Allocates marks per official rubric
- ☑ Generates examiner-style feedback

- ☑ Calculates confidence scores
- ☑ Returns EngineResponse with marking suggestions
- ☑ Does NOT call other engines

2.4.4 Validation & Consistency Engine

Status: ☑ IMPLEMENTED **Files:** `backend/app/engines/ai/validation_consistency/engine.py`

- ☑ Validates AI marking outputs
- ☑ Enforces rubric compliance
- ☑ Checks mark bounds
- ☑ Validates internal consistency
- ☑ Returns `is_valid` flag
- ☑ Orchestrator enforces veto (aborts pipeline if invalid)
- ☑ Does NOT call other engines

2.4.5 Recommendation Engine

Files: `backend/app/engines/ai/recommendation/adapter.py`, `backend/app/engines/ai/recommendation/engine.py`

Status: ☑ IMPLEMENTED (with adapter pattern)

- ☑ Generates personalized study recommendations
- ☑ Uses adapter for orchestrator compatibility
- ☑ Returns EngineResponse
- ⚠ Has standalone FastAPI routes (bypasses orchestrator) - see Section 5

2.4.6 Handwriting Interpretation Engine

Files: `backend/app/engines/ai/handwriting_interpretation/engine.py`

Status: ☑ IMPLEMENTED

- ☑ OCR for handwritten answers
- ☑ Returns EngineResponse
- ☑ Does NOT call other engines

2.4.7 AI Routing & Cost Control Engine

Files: `backend/app/engines/ai/ai_routing_cost_control/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Routes AI requests to optimize cost
- ☑ Cache-first strategy
- ☑ Model selection (OSS vs paid)
- ☑ Returns EngineResponse

2.4.8 Topic Intelligence Engine

Files: `backend/app/engines/ai/topic_intelligence/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Finds related topics
- ☑ Returns EngineResponse
- ☑ Does NOT call other engines

2.5 Engine Contracts

Files: `backend/app/contracts/engine_response.py`, `backend/app/contracts/trace.py`

Status: ☑ IMPLEMENTED

- ☑ Standardized `EngineResponse[T]` contract
- ☑ Mandatory `EngineTrace` in all responses
- ☑ Type-safe with Pydantic
- ☑ All engines return EngineResponse

What Works:

- ☑ Consistent response format
- ☑ Trace metadata included
- ☑ Success/error handling
- ☑ Confidence scores

2.6 Audit & Compliance Engine

Files: `backend/app/engines/audit_compliance/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Captures audit evidence
- ☑ Stores immutable audit records
- ☑ Returns EngineResponse
- ☑ Used in appeal reconstruction

2.7 Background Processing Engine

Files: `backend/app/engines/background_processing/engine.py`

Status: ☑ IMPLEMENTED

- ☑ Queue-based job execution
- ☑ Worker pool architecture
- ☑ Retry and idempotency policies
- ☑ Observability metrics
- ☑ Returns EngineResponse

Issues:

- ⚠ Not integrated into main pipelines (only invoked separately)
- ⚠ Background jobs may not propagate trace_id correctly

SECTION 3: Partially Implemented Components

3.1 RAG-Based Marking Flow

What Exists:

- ☑ Embedding engine generates vectors
- ☑ Retrieval engine fetches evidence
- ☑ Reasoning engine marks using evidence
- ☑ Validation engine checks output
- ☑ Pipeline order: `embedding → retrieval → reasoning_marking → validation → results`

What Is Missing:

- ❌ **No explicit data flow validation**: Orchestrator does not verify that `retrieval` output contains `retrieved_evidence` before passing to `reasoning_marking`
- ❌ **No evidence quality checks**: Retrieval engine may return empty evidence, but reasoning engine may not handle this gracefully
- ❌ **No embedding-retrieval coupling validation**: Orchestrator does not verify embedding dimensions match retrieval requirements

Why Incomplete:

- Pipeline assumes engines will handle missing data, but no explicit contract enforcement
- No inter-engine data validation layer

3.2 Traceability & Auditability

What Exists:

- ☑ `ExecutionContext` with `trace_id`, `request_id`, `user_id`
- ☑ Gateway creates trace context
- ☑ Orchestrator propagates context to engines
- ☑ Engines log with trace_id
- ☑ Audit engine stores trace_id

What Is Missing:

- ❌ **Inconsistent trace propagation**: Some engines may not receive context properly (depends on orchestrator implementation)
- ❌ **No distributed tracing**: No correlation IDs across async operations
- ❌ **No trace aggregation**: No centralized trace storage for querying
- ❌ **Background jobs trace isolation**: Background jobs may generate new trace_ids instead of preserving original

Why Incomplete:

- Trace context exists but not enforced at engine contract level
- No observability backend (e.g., OpenTelemetry, Jaeger)

3.3 Async/Background Task Handling

What Exists:

- ☑ Background Processing Engine with queue architecture
- ☑ Worker pool implementation
- ☑ Job types: marking, embedding, analytics, maintenance
- ☑ Retry policies
- ☑ Idempotency enforcement

What Is Missing:

- ❌ **No pipeline integration**: Background engine not invoked from main pipelines
- ❌ **No async job scheduling**: No mechanism to schedule background jobs from orchestrator
- ❌ **No job status tracking API**: Cannot query job status from frontend
- ❌ **No trace propagation to background jobs**: Background jobs may lose trace context

Why Incomplete:

- Background engine exists but is not integrated into the execution flow
- No bridge between orchestrator and background queue

3.4 Data Storage Separation

What Exists:

- ☑ MongoDB used for:
  - Exam structure (subjects, papers, schedules)
  - Results storage
  - Audit records
  - Vector embeddings (MongoDB Atlas)
  - Learning analytics
  - Governance reports
- ☑ SQL mentioned in settings (`DATABASE_URL`) but usage unclear

What Is Missing:

- ❌ **No clear SQL usage**: `DATABASE_URL` configured but no SQL repositories found
- ❌ **No storage separation policy**: No documented rule for when to use SQL vs MongoDB
- ❌ **No vector store isolation**: Vector embeddings in MongoDB but no dedicated vector DB (e.g., Pinecone, Weaviate)

Why Incomplete:

- MongoDB used for everything, SQL configured but unused

- No architectural decision record (ADR) for storage choices

3.5 Engine Input/Output Schemas

What Exists:

- ☑ Pydantic schemas for all engines
- ☑ Input schemas: `*Input` classes
- ☑ Output schemas: `*Output` classes
- ☑ Schema validation in engines

What Is Missing:

- ❌ **No schema registry**: No centralized catalog of engine contracts
- ❌ **No schema versioning**: Engines have version but schemas don't
- ❌ **No backward compatibility checks**: Schema changes may break pipelines
- ❌ **No inter-engine schema validation**: Orchestrator does not validate that engine outputs match next engine's input requirements

Why Incomplete:

- Schemas exist per-engine but no cross-engine validation layer

---

SECTION 4: Missing Components

4.1 Missing Engine: `"identity"`

Required By Pipelines:

- `exam_attempt_v1` (line 102)
- `handwriting_exam_attempt_v1` (line 118)
- `topic_practice_v1` (line 135)

Registry Has: `"identity_subscription"` only

Impact: System will fail on startup validation for these pipelines.

Fix Required: Either:

1. Register `"identity"` as alias to `IdentitySubscriptionEngine`, OR
2. Update pipelines to use `"identity_subscription"`

4.2 Missing: Centralized Schema Registry

Required For:

- Cross-engine contract validation
- API documentation generation
- Frontend contract synchronization

Status: NOT IMPLEMENTED

4.3 Missing: Distributed Tracing Backend

Required For:

- End-to-end trace correlation
- Performance monitoring
- Debugging production issues

Status: NOT IMPLEMENTED

Options: OpenTelemetry, Jaeger, Zipkin

4.4 Missing: Background Job Scheduler Integration

Required For:

- Scheduling async jobs from orchestrator
- Job status tracking
- Job result retrieval

Status: NOT IMPLEMENTED

4.5 Missing: Inter-Engine Data Validation Layer

Required For:

- Validating that engine outputs match next engine's input schema
- Catching data flow errors early
- Ensuring RAG flow integrity

Status: NOT IMPLEMENTED

4.6 Missing: SQL Database Usage

Configured But Unused:

- `DATABASE_URL` in settings
- No SQL repositories found
- No SQLAlchemy models

Status: NOT IMPLEMENTED

Decision Required: Use SQL for transactional data or remove configuration.

SECTION 5: Architectural Violations

5.1 CRITICAL: Pipeline-Registry Engine Name Mismatch

File: `backend/app/orchestrator/pipelines.py` (lines 102, 118, 135)
Violation: Pipelines reference `"identity"` but registry only has `"identity_subscription"`

Rule Violated: Engine registry must contain all engines referenced in pipelines.

Why Dangerous:

- System will fail on startup with `RuntimeError` for missing engine
- Three pipelines (`exam_attempt_v1`, `handwriting_exam_attempt_v1`, `topic_practice_v1`) cannot execute
- Production deployment will fail immediately

Required Fix:

```
# Option 1: Register alias
engine_registry.register("identity", IdentitySubscriptionEngine())

# Option 2: Update pipelines
"exam_attempt_v1": ["identity_subscription", ...]
```

Priority: P0 (Blocks production)

5.2 CRITICAL: Engine Isolation Violation (Intent)

File: `backend/app/engines/core/practice_assembly/engine.py` (lines 106-122)
Violation: Commented-out code shows intent to call orchestrator from within engine

Rule Violated: Engines must never call other engines directly or via orchestrator.

Code:

```
# related_result = await self.orchestrator.execute_engine(
#     engine_name="topic_intelligence",
#     ...
# )
```

Why Dangerous:

- If uncommented, creates circular dependency
- Violates engine isolation principle
- Makes engines dependent on orchestrator (should be reverse)
- Breaks testability (engines cannot be tested in isolation)

Required Fix:

- Remove commented code

- If topic expansion needed, add `topic_intelligence` to pipeline BEFORE `practice_assembly`
- Pass topic expansion results as input to `practice_assembly`

Priority: P1 (Architectural integrity)

5.3 INVALID: Standalone Recommendation Engine Routes

File: `backend/app/engines/ai/recommendation/routes.py`
Violation: FastAPI routes that bypass orchestrator

Rule Violated: All engine execution must go through orchestrator.

Why Dangerous:

- Bypasses pipeline execution
- Skips audit compliance
- No trace_id propagation
- No validation enforcement
- Creates dual execution paths (orchestrator vs direct)

Required Fix:

- Remove standalone routes
- Use orchestrator pipeline `student_dashboard_v1` which includes recommendation engine
- If direct access needed, create gateway endpoint that calls orchestrator

Priority: P1 (Architectural integrity)

5.4 WARNING: Deprecated Direct Engine Execution

File: `backend/app/orchestrator/orchestrator.py` (lines 457-491)
Violation: `execute()` method allows direct engine execution

Rule Violated: All execution must use pipelines.

Status: Marked as deprecated with warning log

Why Dangerous:

- Allows bypassing pipeline definitions
- Skips engine sequence validation
- May be used by mistake in production

Required Fix:

- Remove `execute()` method entirely
- Update any code using it to use `execute_pipeline()` instead

Priority: P2 (Code quality)

5.5 WARNING: Practice Assembly Engine Orchestrator Dependency

File: `backend/app/engines/core/practice_assembly/engine.py` (line 57, 67)
Violation: Engine constructor accepts `orchestrator` parameter

Rule Violated: Engines should not depend on orchestrator.

Why Dangerous:

- Creates tight coupling
- Makes engines non-testable without orchestrator
- Violates dependency inversion principle

Required Fix:

- Remove orchestrator parameter
- Pass topic expansion results as input to engine
- Orchestrator should call `topic_intelligence` before `practice_assembly` in pipeline

Priority: P2 (Code quality)

SECTION 6: AI & RAG Compliance Check

6.1 Is RAG Marking Correctly Implemented? **PARTIALLY**

Flow:

1. ☑ Submission → Embedding (vectorizes answer)
2. ☑ Embedding → Retrieval (searches vector store)
3. ☑ Retrieval → Reasoning (marks using evidence)
4. ☑ Reasoning → Validation (validates marks)
5. ☑ Validation → Results (stores if valid)

What Works:

- ☑ Pipeline order is correct
- ☑ Engines are isolated (no direct calls)
- ☑ Evidence is retrieved before marking
- ☑ Validation can veto invalid marks

What's Missing:

- ❌ No explicit validation that retrieval returned evidence before reasoning
- ❌ No evidence quality threshold enforcement
- ❌ No fallback if retrieval returns empty evidence

Verdict: Flow is correct but lacks defensive checks.

6.2 Is AI Explainable and Evidence-Anchored? **YES**

Evidence Anchoring:

- ☑ Retrieval engine returns evidence with source references
- ☑ Reasoning engine uses evidence in marking
- ☑ Audit engine stores evidence
- ☑ Appeal reconstruction rehydrates evidence

Explainability:

- ☑ Reasoning engine provides feedback with evidence citations
- ☑ Validation engine explains violations
- ☑ Appeal reconstruction provides human-readable explanations

Verdict: ☑ Compliant

6.3 Is Validation Enforceable? **YES**

Enforcement:

- ☑ Validation engine returns `is_valid` flag
- ☑ Orchestrator checks `is_valid` after validation engine (lines 352-400)
- ☑ Pipeline aborts if `is_valid = False` (raises `PipelineExecutionError`)
- ☑ Invalid marks never reach Results engine

Verdict: ☑ Compliant

SECTION 7: Orchestrator Integrity Check

7.1 Does the Orchestrator Fully Control Execution? **YES**

Evidence:

- ☑ All engines registered in registry
- ☑ Orchestrator is only component that calls `engine.run()`
- ☑ Gateway forwards to orchestrator only
- ☑ No engines call orchestrator (except practice_assembly intent violation)

Verdict: ☑ Compliant (with one exception noted)

7.2 Are Engines Bypassing It? **NO** (with exceptions)

Exceptions:

- ❌ Recommendation engine has standalone routes (Section 5.3)
- ⚠ Practice assembly engine has commented orchestrator call (Section 5.2)

Verdict: Mostly compliant, but exceptions must be fixed.

7.3 Is Execution Order Enforced? **YES**

Evidence:

- ☑ Pipelines define immutable engine sequences
- ☑ Orchestrator executes engines in pipeline order (line 229)
- ☑ No dynamic reordering
- ☑ Frontend cannot influence order

Verdict: ☑ Compliant

SECTION 8: Traceability & Auditability

8.1 trace_id Generation

Status: ☑ IMPLEMENTED

- ☑ Gateway creates `ExecutionContext` with `trace_id` (line 74 in gateway.py)
- ☑ `trace_id` is UUID4
- ☑ Generated once per request

Issue:

- ⚠ Background jobs may generate new trace_ids (not verified)

8.2 trace_id Propagation

Status: ⚠ PARTIALLY IMPLEMENTED

- ☑ Orchestrator passes `ExecutionContext` to engines (line 282)
- ☑ Engines receive context in `run()` method
- ☑ Engines log with trace_id

Missing:

- ❌ No verification that all engines actually use trace_id
- ❌ Background jobs may not preserve trace_id
- ❌ No distributed tracing correlation

8.3 Logging Completeness

Status: ☑ IMPLEMENTED

- ☑ Structured logging with trace_id
- ☑ Engine execution logs
- ☑ Pipeline execution logs
- ☑ Error logs with context

Missing:

- ❌ No centralized log aggregation
- ❌ No log retention policy

- ❌ No log query interface

8.4 Audit Readiness (Schools/Regulators)

Status: ⚠️ PARTIALLY READY

What Exists:

- ☑ Audit engine stores immutable records
- ☑ Appeal reconstruction provides explanations
- ☑ Evidence is stored and retrievable
- ☑ Trace_id links all operations

What's Missing:

- ❌ No audit query API for regulators
- ❌ No audit report generation
- ❌ No data retention policy enforcement
- ❌ No GDPR compliance features (data deletion, export)

Verdict: Core audit exists but lacks regulatory interface.

SECTION 9: Next Build Steps (STRICT ORDER)

STEP 1: Fix Pipeline-Registry Engine Name Mismatch

Priority: P0 (Blocks Production)

What: Register `"identity"` engine or update pipelines to use `"identity_subscription"`

Why: System cannot start with current mismatch. Three pipelines will fail.

Done When:

- All pipelines can execute without `RuntimeError` for missing engine
- Startup validation passes
- All three affected pipelines (`exam_attempt_v1`, `handwriting_exam_attempt_v1`, `topic_practice_v1`) execute successfully

Files to Change:

- `backend/app/orchestrator/engine_registry.py` (add alias) OR
- `backend/app/orchestrator/pipelines.py` (update engine names)

STEP 2: Remove Engine Isolation Violations

Priority: P1 (Architectural Integrity)

What:

1. Remove commented orchestrator call from `practice_assembly` engine
2. Remove standalone recommendation engine routes
3. Remove orchestrator parameter from practice_assembly constructor

Why: Violates core architecture principle. Creates technical debt and testability issues.

Done When:

- No engines have orchestrator dependencies
- No engines have commented-out orchestrator calls
- All execution goes through orchestrator pipelines
- All tests pass with isolated engines

Files to Change:

- `backend/app/engines/core/practice_assembly/engine.py`
- `backend/app/engines/ai/recommendation/routes.py` (remove or redirect to gateway)
- Update `topic_practice_v1` pipeline to include `topic_intelligence` before `practice_assembly`

STEP 3: Implement Inter-Engine Data Validation

Priority: P1 (Data Integrity)

What: Add validation layer in orchestrator that checks engine outputs match next engine's input schema

Why: Prevents runtime errors from schema mismatches. Ensures RAG flow integrity.

Done When:

- Orchestrator validates embedding output has correct dimensions before retrieval
- Orchestrator validates retrieval output has evidence before reasoning
- Orchestrator validates reasoning output has marks before validation
- All validations logged with trace_id

Files to Create/Change:

- `backend/app/orchestrator/data_validator.py` (new)
- `backend/app/orchestrator/orchestrator.py` (add validation calls)

STEP 4: Integrate Background Processing into Pipelines

Priority: P1 (Scalability)

What: Add mechanism for orchestrator to schedule background jobs and track status

Why: Enables async execution for long-running tasks without blocking user requests.

Done When:

- Orchestrator can schedule background jobs
- Background jobs preserve original trace_id

- Job status can be queried via API
- Background jobs are integrated into marking pipeline for heavy workloads

Files to Create/Change:

- `backend/app/orchestrator/background_scheduler.py` (new)
- `backend/app/engines/background_processing/engine.py` (add trace_id preservation)
- `backend/app/api/endpoints/job_status_endpoints.py` (new)

STEP 5: Implement Distributed Tracing

Priority: P2 (Observability)

What: Add OpenTelemetry or similar for end-to-end trace correlation

Why: Enables production debugging and performance monitoring.

Done When:

- All requests have distributed trace IDs
- Traces span gateway → orchestrator → engines → background jobs
- Trace data queryable via UI or API
- Performance metrics collected per trace

Files to Create/Change:

- `backend/app/observability/tracing.py` (enhance existing)
- Add OpenTelemetry SDK
- Configure trace exporter

STEP 6: Add Schema Registry and Versioning

Priority: P2 (Contract Management)

What: Centralized registry of engine input/output schemas with versioning

Why: Enables contract validation, API documentation, and frontend synchronization.

Done When:

- All engine schemas registered in central registry
- Schema versions tracked
- Orchestrator validates inter-engine contracts
- Frontend contracts generated from registry

Files to Create:

- `backend/app/contracts/schema_registry.py` (new)
- `backend/app/contracts/versioning.py` (new)

STEP 7: Implement Audit Query API

Priority: P2 (Regulatory Compliance)

What: API endpoints for regulators/schools to query audit trails

Why: Required for regulatory compliance and transparency.

Done When:

- Regulators can query audit records by trace_id, user_id, date range
- Audit reports can be generated and exported
- Data retention policies enforced
- GDPR compliance features (data export, deletion) implemented

Files to Create:

- `backend/app/api/endpoints/audit_endpoints.py` (new)
- `backend/app/engines/audit_compliance/services/report_generator.py` (new)

STEP 8: Clarify Data Storage Strategy

Priority: P3 (Architecture Clarity)

What: Document and implement clear separation between SQL and MongoDB usage

Why: Current state is unclear. Either use SQL for transactional data or remove configuration.

Done When:

- ADR document explains storage choices
- SQL used for transactional data (if needed) OR removed from config
- MongoDB usage documented
- Vector store strategy documented (MongoDB Atlas vs dedicated vector DB)

Files to Create/Change:

- `backend/docs/architecture/storage_strategy.md` (new)
- `backend/app/config/settings.py` (remove SQL if unused)

SUMMARY

System Status: 65% Complete, NOT Production-Ready

Critical Blockers: 2 (P0)

1. Pipeline-registry engine name mismatch
2. Engine isolation violations (intent)

Architectural Violations: 5 total

- 2 Critical (P0/P1)

- 3 Warnings (P2)

Missing Components: 6 major components

Recommendation: Fix P0 issues immediately. System cannot be safely extended until architectural violations are resolved. Once fixed, proceed with P1 items in order.

End of Audit Report